CS4031 – Compiler Construction
Assignment 2
Report

Imama Sarwar (23I-3048)
Menahil Ahmad (23I-0546)
CS-D

1. Introduction: Brief Overview of LL(1) Parsing

LL(1) parsing is a top-down parsing technique that uses a single lookahead token to make parsing decisions. The name LL(1) indicates:

- **First L:** Left-to-right scanning of input
- **Second L:** Leftmost derivation
- **(1):** One token of lookahead

LL(1) parsers are predictive parsers that utilize a parsing table constructed from the grammar's FIRST and FOLLOW sets. For a grammar to be LL(1), it must satisfy the following conditions:

- No ambiguity (each input corresponds to exactly one parse tree)
- No left recursion (direct or indirect)
- The grammar must be left-factored to eliminate common prefixes

When these conditions are met, the parser can deterministically choose which production to apply based on the current non-terminal and the next input token. The key advantage of LL(1) parsing is its efficiency, O(n) time complexity, and its ability to provide meaningful error messages through systematic error recovery mechanisms.

2. Approach

Data Structures

The key data structures used include:

- `LinkedHashMap` (more specifically `Map<String, List<List<String>>>`) to store CFG
- `Map<String, Set<String>>` to store first and follow sets
- `LinkedHashMap` (more specifically `Map<String, List<List<String>>>`) to store the parsing table
- `List<String>` to model the Stack instance used in LL(1) parsing

The `TreeNode` class is also defined to model the parse tree.

Algorithm Implementation Details & Design Decisions

The program pipeline is as follows:

1. **Parse the grammar file and build the corresponding CFG**

The grammar is read from the input file in the format, where each production is on a new line and rules are separated by |

e.g.

```
Start -> First Second
First -> a | @
Second -> b
```

The input grammar is then stored as a CFG object.

2. **Remove direct and indirect left recursion from the grammar (implementation explained in *Indirect Left Recursion Handling Approach*).**

3. **Left Factor the grammar**

   For each production, e.g. A, find the longest prefix shared by 1 or more productions. The productions that share a common prefix are split; the common prefix remains in A while a new rule A' defines the remaining suffixes of the rule.

```
// Build the new production for A: α A' | γ1 | ... | γm
       List<String> newProd = new ArrayList<>(longestPrefix);
      newProd.add(aPrime);
      List<List<String>> newProdsForA = new ArrayList<>();
       newProdsForA.add(newProd);
       newProdsForA.addAll(nonMatching);

       grammar.put(A, newProdsForA);
       grammar.put(aPrime, suffixes);
```

4. **Construct First and Follow Sets**

   a. **First Sets:**

      First Sets are constructed by following the first set of all first occurring symbols in rules until a non-nullable symbol is encountered, or add @ to the first set of the current symbol.

```
Initialize First(A) = {} for all non-terminals A.
```

```
while changes occur:
   for each production A → X1 X2 ... Xn:
      k = 1; Continue = true
      while Continue and k <= n:
        add First(Xk) - {@} to First(A)
        if @ not in First(Xk): Continue = false
        k++
      if Continue == true:        // @ survived through all Xk
        add @ to First(A)
```

b. Follow Sets

Follow sets are constructed using the rules:add $ to Follow(start), add the first set of the following symbol to the follow set while a non-nullable symbol is encountered, or add the follow set of the production non terminal to the follow set under construction

```
Follow(start) = { $ }
 Follow(A)     = {}   for all other non-terminals A

 while changes occur:
   for each production A → X1 X2 ... Xn:
     for each Xi that is a non-terminal:
       compute First(Xi+1 Xi+2 ... Xn)
       add First(trailer) - {@} to Follow(Xi)
       if @ ∈ First(trailer):
         add Follow(A) to Follow(Xi)
       (if i == n the trailer is empty, so First(trailer) = {@},
        which means Follow(A) is always added for the last symbol)
```

5. Construct the parsing table

To construct the parsing table, place the relevant rule under the terminals in the First set of the current non-terminal. If the non-terminal's first set contains @, also place the null-production under the follow set symbol columns.

```
For each production A -> α:
 1. For each terminal a in FIRST(α):
```

```
        Add A -> α to M[A, a]
    2. If ε is in FIRST(α):
        For each terminal b in FOLLOW(A):
            Add A -> α to M[A, b]
        If $ is in FOLLOW(A), add A -> α to M[A, $]
```

6. **If the grammar is LL(1) proceed**
7. **Read in the input string(s)**
8. **For each input string:**
    a. **Parse the string using the stack and record each step, including error recovery**

    The parser utilises a stack to perform LL(1) parsing. When the top of the stack matches the input terminal, it is popped and the input is advanced. Otherwise, the parsing table is consulted to determine which rule to push onto the stack (in reverse order).

```
INITIALIZE:
    push $ onto stack
    push S onto stack
    ip = 0

REPEAT:
    X = top of stack
    a = input[ip]

    if X == $ and a == $:
        ACCEPT

    else if X == a:
        pop X from stack
        advance ip

    else if X is non-terminal:
        if M[X, a] is not empty:
            pop X from stack
            let M[X, a] = X → Y1 Y2 ... Yn
            push Yn, Yn-1, ..., Y1
            output X → Y1 Y2 ... Yn
        else:
            ERROR

    else:
        ERROR
```

```
UNTIL X == $ and a == $
```

Error handling is described in *Error Recovery Strategy.*

b. Generate the GraphViz parse tree

During parsing, every stack replacement results in the generation of child nodes, and a match generates leaf nodes. The tree is stored and displayed.

Indirect Left Recursion Handling Approach

Order non-terminals in order of appearance in the CFG. Before detecting left recursion, the algorithm replaces higher-order non-terminals with all productions from lower-ordered non-terminals.
This exposes all occurrences of indirect left recursion, which is then removed using:

$$A \rightarrow \beta A'$$
$$A' \rightarrow \alpha A' \mid \varepsilon$$

Error Recovery Strategy

The parser can recognise 4 types of errors:

- Empty Table Entry: NT on stack with no production for current token
- Missing Symbol: expected terminal not found in input
- Unexpected Symbol: terminal on stack but different terminal in input
- Premature End: input exhausted but stack still has symbols

The parser either skips input terminals or pops items from the stack, until the synchronising symbol is found; the table contains an entry for the stack-input pair.

The error recovery loop is as follows, where the decision to pop from the stack or skip input is highlighted.

```
while M[X, a] is empty:
        if a == $ or a ∈ Follow(X): pop X
        else: scan (skip a)
```

3. Challenges: Difficulties Faced and Solutions

Challenge 1: Handling Indirect Left Recursion

Difficulty: Grammar 4 presented a complex case of indirect left recursion:

```
Start -> Alpha a | b
Alpha -> Alpha c | Start d | @
```

This creates a cycle: Start → Alpha → Start, making it impossible for a recursive descent parser to terminate. Simply applying direct left recursion removal algorithms would not suffice.

Solution: Implemented a multi-step transformation approach:

- **Order non-terminals:** Arranged non-terminals in order of appearance (Start, Alpha)
- **Replace higher-order non-terminals:** For production Alpha → Start d, replaced Start with all its productions, yielding Alpha → Alpha a d | b d
- **Apply direct left recursion removal:** Transformed Alpha → Alpha c | Alpha a d | b d | @ to eliminate direct left recursion

This systematic approach exposed all indirect left recursion by expanding non-terminals before removal, resulting in an equivalent LL(1) grammar.

Challenge 2: Implementing Robust Error Recovery

Difficulty: The parser needed to handle multiple error types (Empty Table Entry, Missing Symbol, Unexpected Symbol, Premature End) while maintaining the ability to continue parsing and detect subsequent errors. The recovery strategy needed to decide whether to pop from the stack or skip input tokens.

Solution: Implemented a unified recovery loop in the ErrorHandler.recover() method that uses FOLLOW sets as synchronizing tokens:

```
while (!stack.isEmpty() && !stack.top().equals("$")) {
    String X = stack.top();
    String a = input.get(ip[0]);

    // Check if resynchronized
```

```
    if (parsingTable.containsKey(X) && parsingTable.get(X).get(a) != null) {
        return true;  // Found valid table entry
    }
    if (!parsingTable.containsKey(X) && X.equals(a)) {
        return true;  // Terminal match
    }
    Set<String> followX = followSets.get(X);
    boolean inFollow = followX != null && followX.contains(a);

    if (a.equals("$") || inFollow) {
        stack.pop();  // Pop X - synchronize at FOLLOW set
    } else {
        ip[0]++;      // Skip invalid input token
    }
}
```

The parser classifies errors into four distinct types:

- **EMPTY_TABLE_ENTRY:** Non-terminal on stack with no production for current token
- **MISSING_SYMBOL:** Expected terminal not found in input
- **UNEXPECTED_SYMBOL:** Terminal on stack but different terminal in input
- **PREMATURE_END:** Input exhausted but stack still has symbols

Challenge 3: Preserving Parse Tree Integrity During Error Recovery

Difficulty: When errors occurred during parsing, the parse tree needed to maintain structural integrity while accurately representing where and how errors were recovered. The standard expansion and matching operations didn't account for error scenarios.

Solution: Extended the Tree class with specialized methods for error recovery:

```
// Called when a non-terminal is popped during error recovery
public void popError() {
    if (!nodeStack.isEmpty()) {
        TreeNode node = nodeStack.pop();
        TreeNode errorNode = new TreeNode("✗", true, nodeCounter++);
        node.addChild(errorNode);  // Mark with error symbol
```

```
    }
}

// Called when input tokens are skipped during recovery
public void skipToken(String token) {
    if (nodeStack.size() >= 2) {
        TreeNode context = nodeStack.get(nodeStack.size() - 2);
        TreeNode skipped = new TreeNode("?" + token, true, nodeCounter++);
        context.addChild(skipped);  // Add skipped token as child
    }
}
```

The parse tree visualization distinguishes:

- **Error nodes (✗):** Displayed as red circles indicating where recovery popped a non-terminal
- **Skipped tokens (?token):** Displayed as orange boxes showing tokens that were ignored
- **Normal terminals:** Light blue boxes
- **Non-terminals:** Light yellow ellipses

This approach allows the GraphViz output to visually communicate what happened during error recovery, making it easier to debug the parser's behavior.

Challenge 4: Managing Stack and Parse Tree Synchronization

Difficulty: The parser stack and the parse tree node stack needed to stay synchronized during both normal parsing and error recovery. When the parser popped symbols or expanded productions, the tree structure had to reflect these changes consistently.

Solution: Implemented a dual-stack approach where the Tree class maintains a nodeStack that mirrors the parser's stack:

- **During expansion:** The current non-terminal node is popped from nodeStack, child nodes are created and attached, then pushed in reverse order (so the leftmost symbol ends up on top)
- **During matching:** The top node is popped from nodeStack when a terminal is consumed
- **During error recovery:** Special handling ensures the node stack remains synchronized with parser stack operations

```
public void expand(List<String> production, Set<String> nonTerminals) {
    TreeNode parent = nodeStack.pop();  // Remove current NT

    // Create child nodes
    List<TreeNode> childNodes = new ArrayList<>();
    for (String symbol : production) {
        boolean isTerminal = !nonTerminals.contains(symbol);
        TreeNode child = new TreeNode(symbol, isTerminal, nodeCounter++);
        parent.addChild(child);
        childNodes.add(child);
    }

    // Push in reverse so leftmost symbol is on top
    for (int i = childNodes.size() - 1; i >= 0; i--) {
        nodeStack.push(childNodes.get(i));
    }
}
```

4. Test Cases

4.1 Grammars Tested

Four grammars of increasing complexity were tested to validate the LL(1) parser implementation:

Grammar	Description	Key Features Tested
Grammar 1	Linear grammar: Start → First Second First → a | ε Second → b	ε-productions, simple terminal sequences
Grammar 2	Arithmetic expression: Expr → Expr + Term | Term Term → Term * Factor | Factor Factor → (Expr) | id	Left recursion removal, operator precedence, parentheses
Grammar 3	If-then-else: Stmt → if Cond then Stmt | if Cond then Stmt else Stmt | a Cond → b	Dangling-else ambiguity resolution, nested structures

Grammar 4	Indirect left recursion: Start → Alpha a | b Alpha → Alpha c | Start d | ε	Indirect left recursion elimination, ε-propagation

4.2 Valid Input Strings

Each grammar was tested with multiple valid input strings to ensure correct parsing behavior.

Grammar 1 Valid Inputs:

- a b - Standard production
- b - ε-production for First

Grammar 2 Valid Inputs:

- id
- id + id
- id * id
- id + id * id
- (id + id) * id
- id * (id + id) + id
- (id)
- id + id + id

Grammar 3 Valid Inputs:

- a
- if b then a
- if b then a else a
- if b then if b then a else a
- if b then if b then a else a else a

Grammar 4 Valid Inputs:

- a
- b
- c a
- c c a
- d a
- c d a

4.3 Invalid Input Strings

Invalid inputs were categorized by error type to test the parser's error detection and recovery capabilities.

Grammar 1 Invalid Inputs:

Input	Error Type	Description
a	Missing Symbol	Missing 'b' after 'a'
a a b	Unexpected Symbol	Extra 'a' before 'b'
b a	Unexpected Symbol	Extra 'a' after complete parse

Grammar 2 Invalid Inputs:

Missing Symbol Errors:

Input	Description
id + * id	Missing operand between + and *
id * * id	Missing operand between * and *
(id +) * id	Missing operand after +
id + id *	Missing operand at end
* id + id	Missing operand at start

Unexpected Symbol Errors:

Input	Description
) id + id	Unexpected closing parenthesis
id id + id	Consecutive operands
( ) id	Empty parentheses
id + + id	Consecutive operators

Empty Table Entry Errors:

Input	Description
(id + id	Missing closing parenthesis
id + (id * id	Missing closing parenthesis
id + id)	Extra closing parenthesis

Premature End Errors:

Input	Description
id +	Incomplete - missing operand
id *	Incomplete - missing operand
(id	Incomplete - missing parenthesis

Grammar 3 Invalid Inputs

Input	Error Type	Description
if then a	Missing Symbol	Missing condition 'b'
if b a	Missing Symbol	Missing 'then' keyword
if b then	Missing Symbol	Missing statement
if b then a else	Missing Symbol	Missing statement after 'else'
if a then a	Unexpected Symbol	Unexpected condition 'a'
if b then b	Unexpected Symbol	Unexpected statement 'b'
if b else a	Unexpected Symbol	Unexpected 'else' without 'then'
if b then if then a	Empty Table Entry	Missing condition in nested if
if b then if b else a	Empty Table Entry	Missing 'then' in nested if
if b then if b then	Premature End	Missing statement in nested if
if b then a else	Premature End	Missing statement after 'else'

Grammar 4 Invalid Inputs

Input	Error Type	Description
c	Missing Symbol	Missing 'a' after 'c'

c c	Missing Symbol	Missing 'a' after 'c c'
d	Missing Symbol	Missing 'a' after 'd'
a a	Unexpected Symbol	Extra 'a' after parse
b a	Unexpected Symbol	Extra 'a' after parse
c a b	Unexpected Symbol	Extra 'b' after complete
c d	Empty Table Entry	Missing 'a' after chain
c	Premature End	Input ends early

4.4 Indirect Left Recursion Test Cases

Grammar 4 was specifically designed to test indirect left recursion elimination. The original grammar contains the cycle:

Start → Alpha a | b
Alpha → Alpha c | Start d | ε

This creates indirect recursion: Start → Alpha → Start (via Alpha → Start d)

4.5 Error Recovery Demonstrations

The following test cases demonstrate the parser's error recovery mechanisms across different error types.

Test Case 1: Multiple Error Recovery (Grammar 2)
Input: id + *) id

Step	Stack	Input Remaining	Action
1	$ Expr	id + *) id	Expand Expr → Term → Factor → id
2	$ + Term	+ *) id	Match '+'

3	$ Term	*) id	ERROR: Missing operand before '*'
			Recovery: Skip '*', continue
4	$ Term	) id	ERROR: No production for Term with ')'
			Recovery: Pop Term (')' ∈ FOLLOW(Term))
5	$ Expr	id	Expand Expr → Term → Factor → id
			Result: Parse successful with 2 errors recovered

Test Case 2: Nested Error Recovery (Grammar 3)
Input: if b then if then a

Step	Stack	Input Remaining	Action
1	$ Stmt	if b then if then a	Expand Stmt → if Cond then Stmt
2	$ Stmt then Cond if	b then if then a	Cond → b (match)
3	$ Stmt then if	then if then a	Match 'then.'
4	$ Stmt if	if then a	Expand Stmt → if Cond then Stmt
5	$ Stmt then Cond if if	then a	ERROR: Expected 'b' for Cond
			Recovery: Skip 'then', insert missing 'b'
			Result: Parse continues with nested recovery

Test Case 3: Premature End Recovery (Grammar 4)
Input: c c

Step	Stack	Input Remaining	Action
1	$ Start	c c $	Expand Start → Alpha a
2	$ a Alpha	c c $	Expand Alpha → Alpha c
3	$ a c Alpha	c $	Expand Alpha → Alpha c
4	$ a c c Alpha	c $	ERROR: Expected 'a' but found 'c'
			Recovery: Skip 'c', continue
5	$ a Alpha	$	ERROR: Input ends, stack has 'a'
			Recovery: Pop stack, report premature end

5. Verification

To verify the program output, the results of each step were compared against manual implementations (done by hand) for each grammar and test inputs.
Where applicable, online resources or the textbook were also consulted to verify correctness.

6. Sample Outputs

To demonstrate the working of the program consider the grammar:

```
Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> ( Expr ) | id
```

For the input strings:

String 1 (valid): id + id * id

String 2 (missing symbol): (id +) * id

String 3 (missing/unexpected symbol): id + * id

Grammar Preprocessing and LL(1) Table Construction

```
Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> ( Expr ) | id

--- Removing Left Recursion ---
Expr -> Term Expr'
Term -> Factor Term'
Factor -> ( Expr ) | id
Expr' -> + Term Expr' | @
Term' -> * Factor Term' | @

--- Left Factoring ---
Expr -> Term Expr'
Term -> Factor Term'
Factor -> ( Expr ) | id
Expr' -> + Term Expr' | @
Term' -> * Factor Term' | @

--- First Sets ---
First(Expr) = { (, id }
First(Term) = { (, id }
First(Factor) = { (, id }
First(Expr') = { +, @ }
First(Term') = { *, @ }

--- Follow Sets ---
Follow(Expr) = { $, ) }
Follow(Term) = { +, $, ) }
Follow(Factor) = { *, +, $, ) }
Follow(Expr') = { $, ) }
Follow(Term') = { +, $, ) }

--- LL(1) Parsing Table Construction ---
```

Non-Terminal	(	)	*	+	id	$
Expr	Term Expr'	-	-	-	Term Expr'	-
Term	Factor Term'	-	-	-	Factor Term'	-
Factor	(Expr)	-	-	-	id	-
Expr'	-	@	-	+ Term Expr'	-	@
Term'	-	@	* Factor Term'	@	-	@

```
LL(1) Check: Grammar is LL(1). Each table entry has at most one production
```

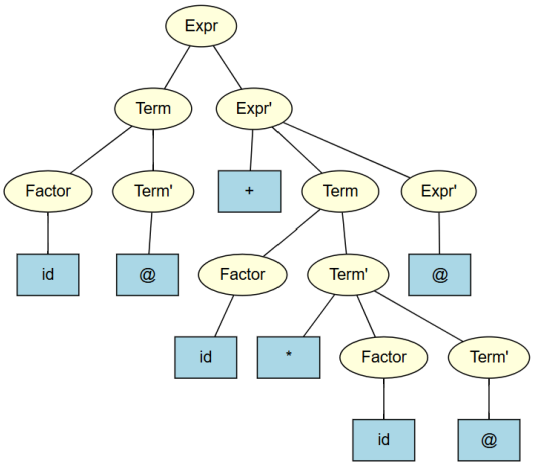
String 1:

```
--- LL(1) Parsing Stack ---

Parsing all input strings...

Parsing: id + id * id
Stack initialized: [$, Expr]
Parse Result: ACCEPT

Parsing Steps:
Step  | Stack Contents                  | Remaining Input      | Action
-------------------------------------------------------------------------------
1     | $ Expr                          | id + id * id $       | Initialize stack with $ and Expr
2     | $ Expr                          | id + id * id $       | Expand Expr -> Term Expr'
3     | $ Expr' Term                    | id + id * id $       | Expand Term -> Factor Term'
4     | $ Expr' Term' Factor            | id + id * id $       | Expand Factor -> id
5     | $ Expr' Term' id               | id + id * id $       | Match: 'id'
6     | $ Expr' Term'                  | + id * id $          | Expand Term' -> @
7     | $ Expr'                        | + id * id $          | Expand Expr' -> + Term Expr'
8     | $ Expr' Term +                 | + id * id $          | Match: '+'
9     | $ Expr' Term                   | id * id $            | Expand Term -> Factor Term'
10    | $ Expr' Term' Factor           | id * id $            | Expand Factor -> id
11    | $ Expr' Term' id               | id * id $            | Match: 'id'
12    | $ Expr' Term'                  | * id $               | Expand Term' -> * Factor Term'
13    | $ Expr' Term' Factor *         | * id $               | Match: '*'
14    | $ Expr' Term' Factor           | id $                 | Expand Factor -> id
15    | $ Expr' Term' id               | id $                 | Match: 'id'
16    | $ Expr' Term'                  | $                    | Expand Term' -> @
17    | $ Expr'                        | $                    | Expand Expr' -> @
18    | $                             | $                    | ACCEPT: Input successfully parsed!
```



String 2 :

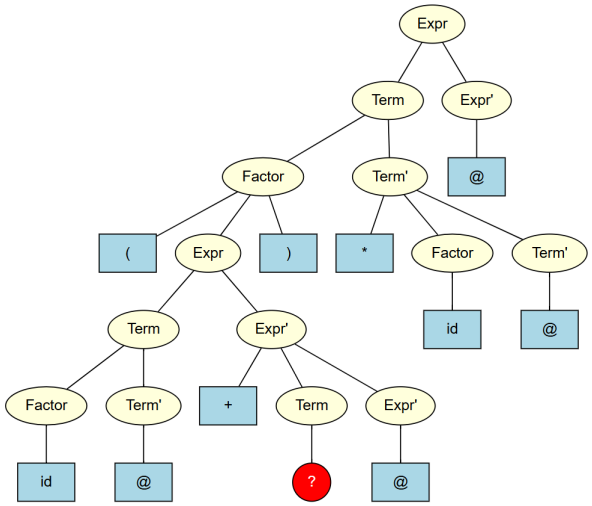
```
--- LL(1) Parsing Stack ---

Parsing all input strings...

Parsing: ( id + ) * id
Stack initialized: [$, Expr]
Parse Result: ACCEPT

Parsing Steps:
Step  | Stack Contents                  | Remaining Input      | Action
--------------------------------------------------------------------------------
1     | $ Expr                          | ( id + ) * id $      | Initialize stack with $ and Expr
2     | $ Expr                          | ( id + ) * id $      | Expand Expr -> Term Expr'
3     | $ Expr' Term                    | ( id + ) * id $      | Expand Term -> Factor Term'
4     | $ Expr' Term' Factor           | ( id + ) * id $      | Expand Factor -> ( Expr )
5     | $ Expr' Term' ) Expr (         | ( id + ) * id $      | Match: '('
6     | $ Expr' Term' ) Expr           | id + ) * id $        | Expand Expr -> Term Expr'
7     | $ Expr' Term' ) Expr' Term     | id + ) * id $        | Expand Term -> Factor Term'
8     | $ Expr' Term' ) Expr' Term' Factor | id + ) * id $        | Expand Factor -> id
9     | $ Expr' Term' ) Expr' Term' id | id + ) * id $        | Match: 'id'
10    | $ Expr' Term' ) Expr' Term'    | + ) * id $           | Expand Term' -> @
11    | $ Expr' Term' ) Expr'          | + ) * id $           | Expand Expr' -> + Term Expr'
12    | $ Expr' Term' ) Expr' Term +   | + ) * id $           | Match: '+'
13    | $ Expr' Term' ) Expr' Term     | ) * id $             | ERROR: Empty Table Entry: No production in M[Term, ')']
14    | $ Expr' Term' ) Expr'          | ) * id $             | Recovery: popped Term
15    | $ Expr' Term' ) Expr'          | ) * id $             | Expand Expr' -> @
16    | $ Expr' Term' )                | ) * id $             | Match: ')'
17    | $ Expr' Term'                  | * id $               | Expand Term' -> * Factor Term'
18    | $ Expr' Term' Factor *         | * id $               | Match: '*'
19    | $ Expr' Term' Factor           | id $                 | Expand Factor -> id
20    | $ Expr' Term' id               | id $                 | Match: 'id'
21    | $ Expr' Term'                  | $                    | Expand Term' -> @
22    | $ Expr'                        | $                    | Expand Expr' -> @
23    | $                             | $                    | ACCEPT: Input successfully parsed!

Errors:
  Empty Table Entry: No production in M[Term, ')']
    Expected: Expected one of: [(, id]
    Found: )
```

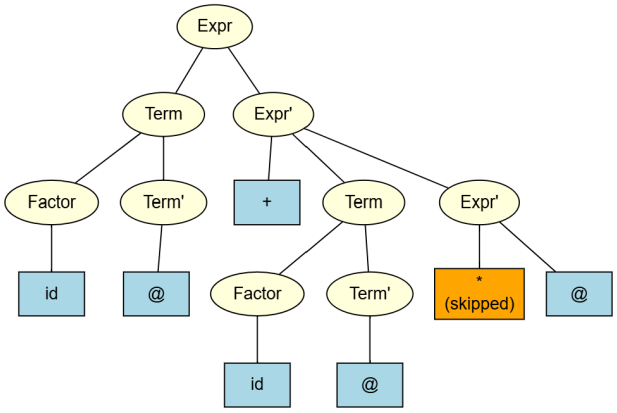


String 3:

```
--------------------------------------------------------------------------------
Parsing: id + * id
Stack initialized: [$, Expr]
DEBUG skipToken(*) nodeStack size=2
  nodeStack[1] = Term
  nodeStack[0] = Expr'
Parse Result: ACCEPT

Parsing Steps:
Step  | Stack Contents                  | Remaining Input      | Action
--------------------------------------------------------------------------------
1     | $ Expr                          | id + * id $          | Initialize stack with $ and Expr
2     | $ Expr                          | id + * id $          | Expand Expr -> Term Expr'
3     | $ Expr' Term                    | id + * id $          | Expand Term -> Factor Term'
4     | $ Expr' Term' Factor           | id + * id $          | Expand Factor -> id
5     | $ Expr' Term' id               | id + * id $          | Match: 'id'
6     | $ Expr' Term'                  | + * id $             | Expand Term' -> @
7     | $ Expr'                        | + * id $             | Expand Expr' -> + Term Expr'
8     | $ Expr' Term +                 | + * id $             | Match: '+'
9     | $ Expr' Term                   | * id $               | ERROR: Empty Table Entry: No production in M[Term, '*']
10    | $ Expr' Term                   | * id $               | Recovery: skipping '*'
11    | $ Expr' Term                   | id $                 | Expand Term -> Factor Term'
12    | $ Expr' Term' Factor           | id $                 | Expand Factor -> id
13    | $ Expr' Term' id               | id $                 | Match: 'id'
14    | $ Expr' Term'                  | $                    | Expand Term' -> @
15    | $ Expr'                        | $                    | Expand Expr' -> @
16    | $                              | $                    | ACCEPT: Input successfully parsed!

Errors:
  Empty Table Entry: No production in M[Term, '*']
    Expected: Expected one of: [(, id]
    Found: *
```



7. Conclusion

Implementing an LL(1) parser with comprehensive error recovery provided deep insight into both the theoretical foundations of parsing and practical implementation challenges. The project demonstrated that

while LL(1) grammars have inherent limitations, requiring left recursion elimination and left factoring, the resulting parsers are efficient, predictable, and capable of providing meaningful error feedback when properly implemented. The most valuable lesson was that error recovery is not merely an add-on feature but a critical component that determines the parser's practical usability; the unified recovery loop using FOLLOW sets as synchronizing tokens proved highly effective for handling empty table entries, missing symbols, unexpected symbols, and premature end conditions.

Additionally, maintaining a dual-stack architecture, one for parsing and one for parse tree construction, simplified the implementation and made the visualization of both successful parses and error recovery points possible through color-coded GraphViz output. The indirect left recursion elimination in Grammar 4 reinforced the importance of systematic non-terminal ordering and expansion before applying direct left recursion removal. Overall, this project reinforced that a well-designed parser is not just about accepting valid inputs, but about failing gracefully with clear, actionable feedback when inputs are invalid.